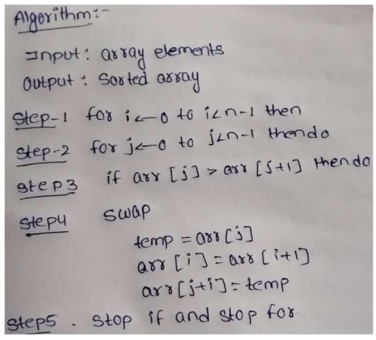
Practical – 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1)Bubble Sort:

Aim:To implementation of timecomplexity of bubble sort.

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main() {

   int n;


   cout << "Enter the number of elements: ";

   cin >> n;


   int arr[n];


   cout << "Enter the elements: ";
```

Name:B.Vennela
Enrollment no:92460118232

```
    for (int i = 0; i < n; i++) {
        cin >> arr[i];

}

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

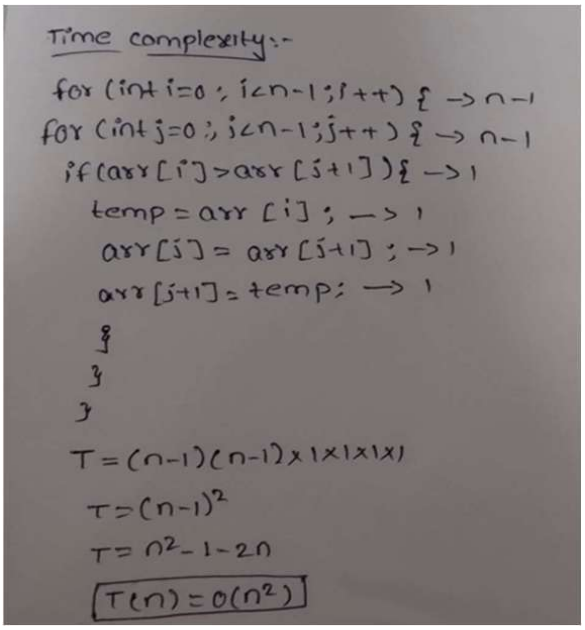
    return 0;
}
```

Output:-

```
Enter the number of elements: 4
Enter the element at index 0: 2
Enter the element at index 1: 6
Enter the element at index 2: 7
Enter the element at index 3: 5

Sorted array:
Index 0 = 2
Index 1 = 5
Index 2 = 6
Index 3 = 7
```

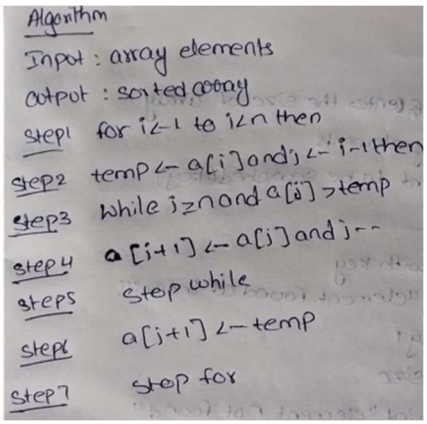
Time complexity:



2) Selection sort:

Aim: To implementation and time complexity of selection sort.

Algorithm:



Program:

```
#include <iostream>

using namespace std;


int main() {

   int n;


   cout << "Enter the number of elements: ";

   cin >> n;


   int arr[n];


   cout << "Enter the elements:" << endl;


   for (int i = 0; i < n; i++) {

      cout << "Enter element at index " << i << ": ";

      cin >> arr[i]
```

```
// Selection Sort
  for (int i = 0; i < n - 1; i++) {
     int minIndex = i;

     for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
           minIndex = j;
        }
     }

     // Swap
     int temp = arr[i];
     arr[i] = arr[minIndex];
     arr[minIndex] = temp;
  }

  cout << "\nSorted array: ";

  for (int i = 0; i < n; i++) {
     cout << arr[i] << " ";
  }

  return 0;
```

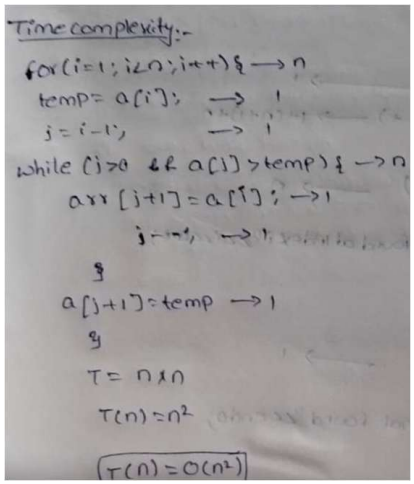
Name:B.Vennela
Enrollment no:92460118232

Output:

```
Enter the number of elements: 4
Enter the elements:
Enter element at index 0: 3
Enter element at index 1: 6
Enter element at index 2: 9
Enter element at index 3: 7
Sorted array: 3 6 7 9

=== Code Execution Successful ===
```

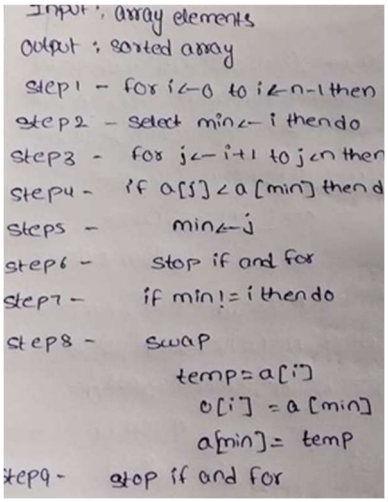
Timecomplexity:



3) Insertion sort:

Aim: To Implementation and Time analysis of Insertion sort

Algorithm:



Name:B.Vennela
Enrollment no:92460118232

Program:

```
#include <iostream>

using namespace std;


int main() {

   int n;


   cout << "Enter the number of elements: ";

   cin >> n;


   int arr[n];


   cout << "Enter the elements:" << endl;


   for (int i = 0; i < n; i++) {

      cout << "Enter element at index " << i << ": ";

      cin >> arr[i];

   }


   // Insertion Sort

   for (int i = 1; i < n; i++) {

      int key = arr[i];

      int j = i - 1;


      while (j >= 0 && arr[j] > key) {
```

```
    arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

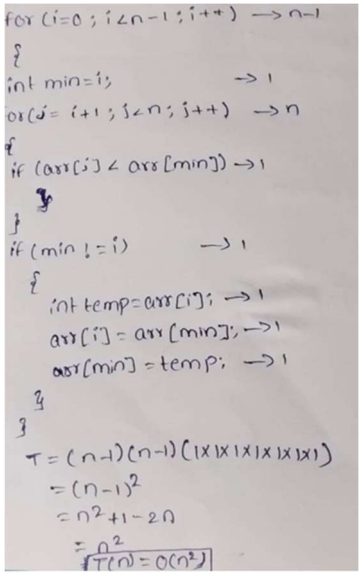
    return 0;
}
```

Output:

```
Enter the number of elements: 3
Enter the elements:
Enter element at index 0: 5
Enter element at index 1: 7
Enter element at index 2: 2
Sorted array: 2 5 7

=== Code Execution Successful ===
```

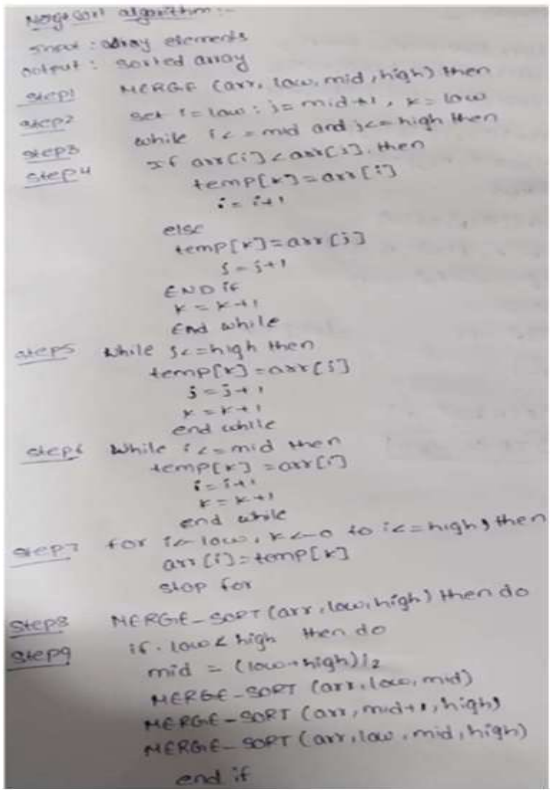
Timecomplexity:



4)Merge Sort:

Aim: To implementation and time analysis of Merge sort.

Algorithm:



Name:B.Vennela
Enrollment no:92460118232

Program:

```
#include <iostream>

using namespace std;


void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;


    int L[n1], R[n2];


    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];


    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];


    int i = 0, j = 0, k = l;
while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j]
  j++;
```

Name:B.Vennela
Enrollment no:92460118232

```
  }
      k++;
  }

  while (i < n1) {
      arr[k] = L[i];
      i++;
      k++;
  }

  while (j < n2) {
      arr[k] = R[j];
      j++;
      k++;
  }
}
void mergeSort(int arr[], int l, int r) {
  if (l < r) {
      int m = l + (r - l) / 2;

      mergeSort(arr, l, m);
      mergeSort(arr, m + 1, r);

      merge(arr, l, m, r);
  }
```

```
}

int main() {
    int n;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter the elements:" << endl;

    for (int i = 0; i < n; i++) {
        cout << "Enter element at index " << i << ": ";
        cin >> arr[i];
    }

    mergeSort(arr, 0, n - 1);
 cout << "\nSorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
 return 0;
}
```

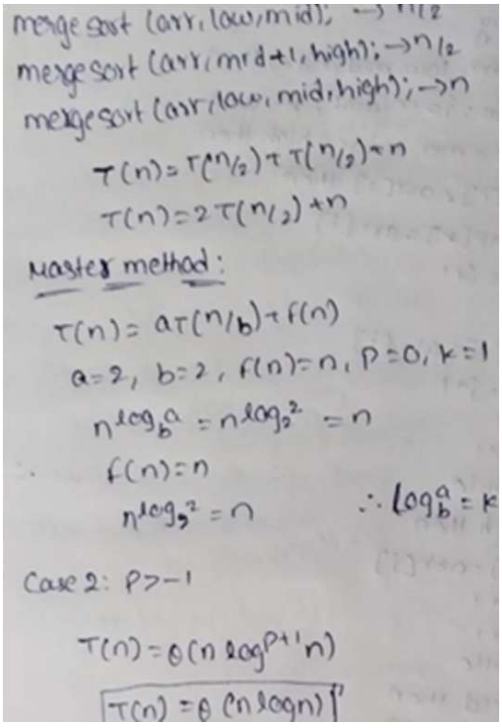
Name:B.Vennela
Enrollment no:92460118232

Output:

```
Enter the number of elements: 6
Enter the elements: 5
8
7
4

2
9
Sorted array: 2 4 5 7 8 9
```

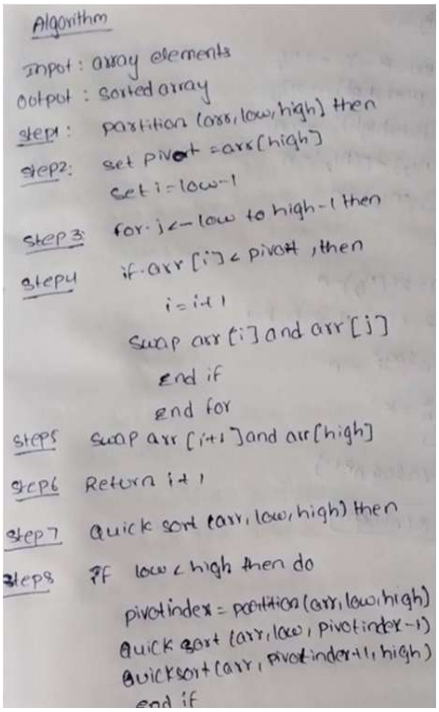
Timecomplexity:



5) Quick sort:

Aim:To implementation of time complexity of quick sort.

Algorithm:



Program:

```
#include <iostream>

using namespace std;


void quickSort(int arr[], int low, int high)

{

   if (low < high)

   {

      int pivot = arr[high];

      int i = low - 1;


      for (int j = low; j < high; j++)

       {
```

```
        if (arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    int pi = i + 1;

     quickSort(arr, low, pi - 1);
    quickSort(arr, pi + 1, high);
  }
}

int main()
{
```

```
int arr[100], n;

  cout << "Enter the number of elements: ";
  cin >> n;

  cout << "Enter the elements:" << endl;

  for (int i = 0; i < n; i++)
  {
     cout << "Enter element at index " << i << ": ";
     cin >> arr[i];
  }

  quickSort(arr, 0, n - 1);

  cout << "\nSorted array: ";

  for (int i = 0; i < n; i++)
  {
     cout << arr[i] << " ";
  }

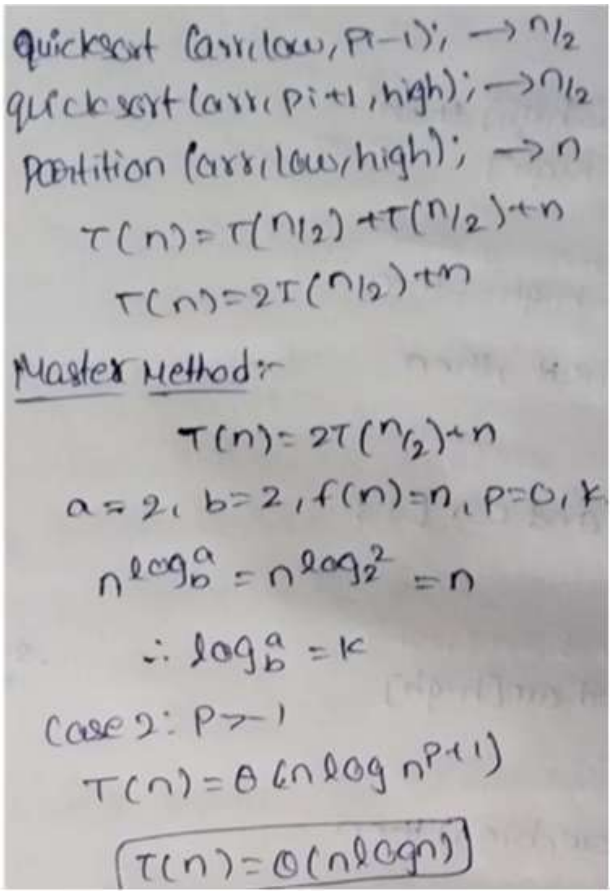
  return 0;
```

Name:B.Vennela
Enrollment no:92460118232

Output:

```
Enter the number of elements: 5
Enter the elements:
Enter element at index 0: 3
Enter element at index 1: 6
Enter element at index 2: 9
Enter element at index 3: 7
Enter element at index 4: 1
Sorted array: 1 3 6 7 9
```

Time complexity:



Name:B.Vennela
Enrollment no:92460118232